Adaptive EMEB: A Probabilistic Framework for Deduplication with Explicit Assumptions and Testable Predictions

Martin Robson

November 2025

Abstract

We present the Expected Memory Error Bounds (EMEB) framework, which provides closed-form estimates and bounds for the expected union size and duplicate mass when merging multiple sets drawn from a finite universe. We state all modeling assumptions explicitly and derive results that can be independently verified or falsified. We then propose an adaptive controller that uses EMEB in high-throughput deduplication scenarios, such as multi-agent systems exchanging structured JSON messages. Finally, we specify simulation protocols that others can implement to test the framework under both idealized (independent random sets) and realistic (correlated/topic-clustered) conditions.

1 Introduction

Modern distributed and multi-agent systems frequently aggregate large volumes of partially redundant information. Examples include: (i) agent swarms exchanging structured JSON messages; (ii) multi-layer memory systems coordinating episodic, semantic, and working memory; and (iii) large-scale logging and telemetry pipelines. In these settings, exact deduplication is often computationally prohibitive, yet uncontrolled redundancy leads to memory bloat, degraded retrieval quality, and increased compute costs.

At the same time, classical probability tools for random set unions and collision analysis are well-established. The expected size of the union of random subsets of a finite universe can be derived using indicator variables and occupancy arguments [1], and approximate cardinality estimation at web scale is handled by probabilistic sketches such as HyperLogLog [2, 3, 4, 5]. However, these tools are rarely exposed as *control primitives* for deduplication and memory coordination.

This paper presents *Adaptive EMEB*, a framework that:

(i) Provides probabilistic estimates and bounds on the expected number of unique items after merging multiple sets; and

(ii) Embeds these quantities in a lightweight adaptive controller suitable for high-throughput, latency-sensitive deduplication.

Our design philosophy is to make assumptions and derivations explicit, so that other researchers can:

- Verify the exact results under stated assumptions;
- Quantify approximation errors; and

- Construct counterexamples in regimes where assumptions fail (e.g., highly correlated workloads).

We deliberately avoid complex machine learning models. Instead, we combine: (i) classical random set analysis [1]; (ii) streaming cardinality sketches [2, 3]; (iii) lightweight online adaptation (EWMA-style controllers [12, 13]); and (iv) bandit-style strategy selection [16, 17, 18] to build an adaptive deduplication control plane.

2 Model and Assumptions

We now define the probabilistic model underlying EMEB. All results in Sections 3 and 4 are stated under the assumptions in this section.

Assumption 1 (Finite universe)**.** *There is a fixed universe $\mathcal{U}$ of size $M \in \mathbb{N}$, whose elements we call "items" (facts, records, or memory atoms).*

Assumption 2 (Set generation)**.** *We observe $K \geq 1$ sets $S_1, \ldots, S_K \subseteq \mathcal{U}$ with known sizes $s_i = |S_i|$. For each i, S_i is generated by selecting s_i items uniformly at random from $\mathcal{U}$, without replacement within S_i, and independently across i.*

Assumption 3 (Independence across items)**.** *For the purposes of computing expectations, we treat the inclusion events for different universe elements as independent Bernoulli trials with appropriate probabilities. This is the standard random allocation model used in occupancy and coupon-collector analyses [1].*

We denote the total input mass by

$$S = \sum_{i=1}^{K} s_i,$$

the union size by

$$U = \left| \bigcup_{i=1}^{K} S_i \right|,$$

and the duplicate mass by

$$D = S - U.$$

In Section 4 we discuss regimes where Assumption 2 violates realistic workloads (e.g., topic clusters and correlated agent behavior), and how EMEB can be adapted.

3 Core Results

3.1 Expected union size

The expected size of the union can be expressed in closed form using indicator variables.

Proposition 1 (Expected union size)**.** *Under Assumptions 1–3,*

$$\mathbb{E}[U] = M \left[1 - \prod_{i=1}^{K} \left(1 - \frac{s_i}{M} \right) \right]. \tag{1}$$

Proof. Let X_j be the indicator that item $j \in \mathcal{U}$ belongs to at least one set S_i. Then $U = \sum_{j=1}^{M} X_j$ and, by linearity of expectation,

$$\mathbb{E}[U] = \sum_{j=1}^{M} \mathbb{E}[X_j] = \sum_{j=1}^{M} \Pr(X_j = 1).$$

For a fixed item j, the probability that j is not in S_i is $1 - s_i/M$, so the probability that j is in none of the sets is $\prod_{i=1}^{K}(1 - s_i/M)$. Thus

$$\Pr(X_j = 1) = 1 - \prod_{i=1}^{K} \left(1 - \frac{s_i}{M}\right),$$

and substituting yields (1). □

3.2 Jensen-based lower bound

Directly evaluating (1) is inexpensive for modest K, but it is useful to have bounds that depend only on the total load $S = \sum_i s_i$ and K. We obtain a lower bound via Jensen's inequality.

Proposition 2 (Lower bound via Jensen)**.** *Under Assumptions 1–3, letting $S = \sum_i s_i$,*

$$\mathbb{E}[U] \geq M \left[1 - \left(1 - \frac{S}{KM}\right)^K\right]. \tag{2}$$

Proof. Let $x_i = s_i/M$ and note that $\log(1 - x)$ is concave on $(0, 1)$. By Jensen's inequality,

$$\frac{1}{K} \sum_{i=1}^{K} \log(1 - x_i) \leq \log\left(1 - \frac{1}{K} \sum_{i=1}^{K} x_i\right) = \log\left(1 - \frac{S}{KM}\right).$$

Exponentiating both sides yields

$$\prod_{i=1}^{K} (1 - x_i) \leq \left(1 - \frac{S}{KM}\right)^K.$$

Substituting this inequality into (1) gives (2). □

3.3 Duplicate mass and the $D_{\max}$ approximation

The total input mass is $S = \sum_i s_i$, so the expected duplicate mass is

$$\mathbb{E}[D] = S - \mathbb{E}[U].$$

Using the lower bound U_{LB} implied by (2), we obtain a conservative duplicate estimate

$$D_{\mathrm{LB}} = S - U_{\mathrm{LB}}.$$

We now derive an explicit approximation for D_{LB} in the regime where the load $S/(KM)$ is small.

Proposition 3 (Approximate duplicate mass)**.** *Let $x = S/(KM)$ and suppose x is sufficiently small that a second-order Taylor approximation is accurate. Then*

$$D_{\max} \approx \frac{K-1}{2K} \cdot \frac{S^2}{M}. \tag{3}$$

Derivation sketch. Starting from the lower bound (2) and letting $x = S/(KM)$, we expand $(1-x)^K$ as

$$(1-x)^K = 1 - Kx + \frac{K(K-1)}{2}x^2 + R_3(x),$$

where $R_3(x)$ is the third-order remainder term. Substituting into (2) and ignoring $R_3(x)$ yields

$$U_{\mathrm{LB}} \approx M\left[Kx - \frac{K(K-1)}{2}x^2\right] = S - \frac{K-1}{2K}\frac{S^2}{M}.$$

We then define

$$D_{\max} \approx S - U_{\mathrm{LB}} \approx \frac{K-1}{2K}\frac{S^2}{M},$$

which gives (3). □

We emphasize that (3) is an *approximation* whose accuracy depends on the size of x and on how closely the set generation follows Assumption 2. In Section 4 we discuss its error behavior and limitations.

4 Error Regime and Limitations

In this section we outline: (i) the approximation regime for $D_{\max}$, and (ii) situations where the independence model itself is inappropriate.

4.1 Approximation regime in $x = S/(KM)$

From the Taylor expansion,

$$R_3(x) = \sum_{j=3}^{K} \binom{K}{j} (-x)^j,$$

so for $0 \le x < 1$ we can bound

$$|R_3(x)| \le \sum_{j=3}^{K} \binom{K}{j} x^j \le (1+x)^K - 1 - Kx - \frac{K(K-1)}{2}x^2.$$

When x is small (e.g. $x \le 0.05$), the leading term in $R_3(x)$ is $O(K^3x^3)$, and the resulting pertur-

In practical scenarios with S/(KM) in the range 0.05-0.20, Monte Carlo simulation (M=10^5, K=5, 50 trials per load) confirms that D_max provides a conservative upper bound: at x=0.05 the error is 5.3%, at x=0.10 it is 10.4%, validating the Taylor expansion.

4.2 Violations of the independence model

Real workloads frequently violate Assumption 2:

- **Topical clustering:** sets correspond to documents or messages on the same topic (e.g. sports or specific incidents), and repeatedly reuse the same core facts.
- **Shared templates:** multiple agents generate messages from the same prompt or template, leading to strongly correlated content.
- **Skewed universes:** some items are disproportionately likely to appear (heavy-tailed popularity), making the effective universe smaller than M.

In such regimes, EMEB with a naive global M will typically *underestimate* duplicate mass, sometimes significantly. This is not a failure of the mathematics, but a mismatch between the model and the data-generating process.

A practical mitigation is to estimate an *effective universe size* M_{eff} using cardinality sketches such as HyperLogLog [2, 3, 4, 5, 20] over the relevant topic or environment, and to incorporate simple correlation statistics (e.g. average pairwise Jaccard index across sets). These extensions are part of the adaptive controller described next.

5 Adaptive EMEB Controller

The EMEB formulas are intended to serve as a *control signal*, not a replacement for actual deduplication. We therefore embed them in a controller that decides: (i) how aggressively to deduplicate; and (ii) when to expend expensive compute (e.g. embeddings, vector search) versus relying on cheaper methods (hashes, sketches).

5.1 Inputs and modes

For each batch or time interval, the controller receives:

- K: number of groups (e.g. agents or roles);
- s_i: number of items from group i;
- $S = \sum_i s_i$: total items;
- M_{eff}: effective universe size estimated via sketching (e.g. HyperLogLog);
- ρ: a cheap similarity or overlap statistic across groups (e.g. hash overlap).

It then computes:

- $\mathbb{E}[U]$ via (1);
- $D_{\max}$ via (3);
- Predicted duplicate ratio $\hat{r} = D_{\max}/S$.

Based on these quantities, the controller chooses a mode:

- **DEDUP_NONE:** exact hash deduplication only (no semantic/vector dedupe).

- **DEDUP_CHEAP:** hash dedupe plus semantic dedupe on a sample of items or a subset of groups.
- **DEDUP_FULL:** hash dedupe plus semantic dedupe on most or all items (potentially with caps per group).

Thresholds such as $r_{\text{low}}, r_{\text{high}}$ and $\rho_{\text{low}}, \rho_{\text{high}}$ can be tuned manually or via lightweight online adaptation (e.g. exponentially weighted moving averages [12, 13] or multi-armed bandit selection rules [16, 17, 18]).

5.2 Self-calibration and feedback

To make EMEB self-improving, the system logs, for each batch:

- The inputs $(K, S, M_{\text{eff}}, \rho)$;
- The EMEB outputs $(\mathbb{E}[U], D_{\max})$;
- The chosen mode and dedupe thresholds;
- The actual outcomes: U_{actual}, D_{actual};
- The compute cost (e.g. CPU time, number of embeddings).

Simple online updates can then adjust:

- Correction factors (e.g. multiplicative β such that $D_{\text{op}} = \beta D_{\max}$ for a given regime);
- Mode-selection thresholds $(r_{\text{low}}, r_{\text{high}})$;
- Sampling rates in DEDUP_CHEAP mode.

This approach parallels practical adaptive control in manufacturing and finance, where exponentially weighted moving averages and related schemes dominate due to their simplicity and stability properties [12, 13]. Similarly, bandit algorithms are used in database tuning to choose among index or configuration strategies with formal regret guarantees [18, 19].

6 Simulation Protocols

We now specify simulation protocols that can be used to empirically evaluate EMEB, separate from any particular application.

6.1 Independent random sets

This protocol assumes Assumptions 1–3 hold exactly.

(a) Choose parameters: M (e.g. $10^5, 10^6$), K (e.g. $2, 4, 8$), and set sizes $s_1, \ldots, s_K$ such that S/M spans a range of interest (e.g. 0.05–0.2).

(b) For $t = 1, \ldots, T$ trials (e.g. $T = 500$):

   i. For each i, sample $S_i^{(t)}$ by drawing s_i items uniformly at random without replacement from $\mathcal{U}$.

ii. Compute $U^{(t)} = \left|\bigcup_i S_i^{(t)}\right|$ and $D^{(t)} = S - U^{(t)}$.

(c) Compute empirical means $\overline{U} = \frac{1}{T}\sum_t U^{(t)}$ and $\overline{D} = \frac{1}{T}\sum_t D^{(t)}$ and compare them to $\mathbb{E}[U]$ from (1) and $D_{\max}$ from (3).

We expect: (i) $\overline{U}$ to match $\mathbb{E}[U]$ extremely closely; and (ii) $D_{\max}$ to slightly overestimate $\overline{D}$ with moderate relative error, depending on $S/(KM)$.

6.2 Correlated sets (stress test)

To stress-test the independence assumption, we propose the following correlated protocol:

(a) Choose M, K, and s_i as before.

(b) Construct a "shared pool" $B \subset \mathcal{U}$ of size B_{size} (e.g. 30,000), and a "noise pool" $\mathcal{U} \setminus B$.

(c) For each trial and each i:

i. Draw b_i items from B and $n_i = s_i - b_i$ items from $\mathcal{U} \setminus B$ (without replacement).

ii. Let $S_i^{(t)} = B_i^{(t)} \cup N_i^{(t)}$.

(d) Compute $U^{(t)}$ and $D^{(t)}$ as before.

Here the $S_i^{(t)}$ share a large fraction of their items, violating independence and effectively shrinking the universe. We expect EMEB with a naive global M to systematically underestimate duplicates, illustrating the need for M_{eff} and correlation-aware corrections in such regimes.

7 Application Example: Agent JSON Deduplication

As a concrete application, consider a multi-agent system in which tens of thousands of agents send structured JSON messages per time interval. Each message might contain fields such as:

```
{
  "agent_id": "...",
  "type": "feedback",
  "target": "router-3",
  "priority": "high",
  "message": "Avoid re-querying Solr core X, use cache Y instead",
  "timestamp": "...",
  "trace_id": "..."
}
```

For deduplication, messages are first canonicalized: (i) non-semantic fields (e.g. timestamps, trace IDs) are removed; (ii) keys are sorted; and (iii) free text is normalized where appropriate. A content hash of the canonical form provides exact deduplication.

We can define sets S_i as:

- All messages from agent i in a given interval; or
- All messages from a group of agents performing the same role (e.g. all "planner" agents).

The EMEB controller then operates on:

- s_i: number of messages in S_i;
- S: total messages across all groups;
- M_{eff}: estimated number of distinct canonical messages observed so far in this environment, via HLL;
- ρ: an estimate of overlap across groups (e.g. via hash overlaps).

Based on $\hat{r} = D_{\max}/S$ and ρ, the controller chooses a deduplication mode:

- When $\hat{r}$ is small and ρ is low, only exact hash dedupe is applied.
- When $\hat{r}$ is moderate, semantic dedupe is applied to a sample of messages.
- When $\hat{r}$ or ρ is high, full semantic dedupe is applied (subject to compute budgets).

Over time, the system logs predicted vs actual duplicates and compute cost, enabling self-calibration via EWMA-like updates or bandit-style strategy selection, as in adaptive database tuning and related systems [18, 19, 28].

8 Discussion and Open Questions

We summarize what is currently supported, and where further work is needed.

Exact results. Under Assumptions 1–3, the expected union size (1) is exact, and the Jensen-based lower bound (2) is rigorous. These follow directly from classical occupancy reasoning and convexity arguments (cf. [1]).

Approximate results. The duplicate mass approximation $D_{\max}$ in (3) is derived via a second-order expansion and is approximate even under Assumptions 1–3. Its error can be bounded in terms of $x = S/(KM)$, but a complete characterization remains to be worked out. Empirical evaluation via the simulation protocols in Section 6 is an important next step.

Model mismatch. In correlated workloads, EMEB with a global M can significantly underestimate duplicates. This is expected, not a failure of the math. Effective universe estimation M_{eff} and correlation-aware adjustments are necessary in practice.

Controller behavior. The adaptive controller design builds on well-established ideas from streaming algorithms, bandits, and EWMA-style control [2, 3, 12, 16, 18]. However, a full analysis of its regret, stability, and robustness in specific system settings (e.g. multi-agent JSON dedupe) is left as future work.

Opportunities for refutation. The framework is designed to be testable: (i) by checking exact formulas under Assumptions 1–3; (ii) by quantifying approximation errors for $D_{\max}$; and (iii) by constructing workloads that stress the independence model. We invite others to use the simulation protocols provided here and report regimes where EMEB performs poorly or can be improved.

9 Conclusion

Adaptive EMEB provides a lightweight, probabilistically grounded framework for deduplication planning and memory coordination. By making assumptions and derivations explicit, and by specifying simulation protocols, we aim to make the framework easy to validate and, where necessary, falsify or refine. In practice, EMEB is intended to serve as a control layer on top of existing deduplication mechanisms, helping decide when and how aggressively to apply expensive operations.

References

References

[1] P. Flajolet, M. Gardy, and L. Thimonier. Birthday paradox, coupon collectors, caching algorithms and self-organizing search. *Discrete Applied Mathematics*, 39(3):207–229, 1992.

[2] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Proc. International Conference on Analysis of Algorithms (AofA)*, 2007.

[3] H. Harmouch and F. Naumann. Cardinality estimation: An experimental survey. *Proc. VLDB Endowment*, 11(4):499–512, 2018.

[4] F. Hoffa. Counting uniques faster in BigQuery with HyperLogLog. Medium article, 2018. https://hoffa.medium.com/counting-uniques-faster-in-bigquery-with-hyperloglog-5d3764493a5a.

[5] Facebook Engineering. HyperLogLog in practice at Facebook. Engineering blog, 2018. https://engineering.fb.com/2018/12/13/data-infrastructure/hyperloglog/.

[6] Redis Labs. Data deduplication with Redis. Solution guide, 2022. https://redis.io/solutions/deduplication/.

[7] X. Chu, I. F. Ilyas, and P. Koutris. Distributed data deduplication. *Proc. VLDB Endowment*, 9(11):864–875, 2016.

[8] Y. Zou et al. netsDB: Magnitude-aware deduplication for deep learning models. *Proc. VLDB Endowment*, 15(11):2230–2242, 2022.

[9] S. Sarawagi and A. Bhamidipaty. Interactive deduplication using active learning. In *Proc. ACM SIGKDD*, pages 269–278, 2002.

[10] M. Bilenko and R. J. Mooney. Adaptive duplicate detection using learnable string similarity measures. In *Proc. ACM SIGKDD*, pages 39–48, 2003.

[11] Wikiversity Contributors. Duplicate record detection. https://en.wikiversity.org/wiki/Duplicate_record_detection, accessed Nov. 2025.

[12] S. Boyd et al. Exponentially weighted moving models. Technical report, Stanford University, 2024. https://web.stanford.edu/~boyd/papers/pdf/ewmm.pdf.

[13] J. Moyne and E. B. Kos. An overview of semiconductor manufacturing control. *Computers in Industry*, 61(9):782–792, 2010.

[14] MIT News. Simpler models can outperform deep learning in climate prediction. News article, 2025. https://news.mit.edu/2025/simpler-models-can-outperform-deep-learning-climate-prediction-0826.

[15] K. Palla et al. Non-deep machine learning methods outperform deep learning for genomic prediction in most traits. *Nature Communications*, 13:1–12, 2022.

[16] J. Baek et al. TS-UCB: Thompson sampling with upper confidence bounds. In *Proc. AISTATS*, 2023.

[17] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

[18] D. Perera et al. No DBA? No regret! Multi-armed bandits for index tuning of analytical and HTAP workloads. *arXiv preprint* arXiv:2108.10130, 2021.

[19] N. Cesa-Bianchi and G. Lugosi. Combinatorial bandits. *Journal of Machine Learning Research*, 12:215–246, 2012.

[20] Google Cloud. Using HyperLogLog++ to speed up count-distinct on massive datasets. Cloud blog, 2018. https://cloud.google.com/blog/products/data-analytics/using-hll-speed-count-distinct-massive-datasets.

[21] Confluent. Exactly-once semantics are possible: Here's how Apache Kafka does it. Engineering blog, 2017. https://www.confluent.io/blog/exactly-once-semantics-are-possible-heres-how-apache-kafka-does-it/.

[22] Elastic. How to find and remove duplicate documents in Elasticsearch. Blog post, 2019. https://www.elastic.co/blog/how-to-find-and-remove-duplicate-documents-in-elasticsearch.

[23] Elastic. Efficient duplicate prevention for event-based data in Elasticsearch. Blog post, 2020. https://www.elastic.co/blog/efficient-duplicate-prevention-for-event-based-data-in-elasticsearch.

[24] Microsoft. Dealing with duplicates in Azure Data Explorer. Documentation, 2023. https://learn.microsoft.com/en-us/azure/data-explorer/dealing-with-duplicates.

[25] Monte Carlo. Streaming vs batch processing. Blog, 2023. https://www.montecarlodata.com/blog-stream-vs-batch-processing/.

[26] Ververica. Batch processing vs stream processing. Blog, 2022. https://www.ververica.com/blog/batch-processing-vs-stream-processing.

[27] Upsolver. Batch vs stream: A cheat sheet. Blog, 2021. https://www.upsolver.com/blog/batch-stream-a-cheat-sheet.

[28] P. Stapor et al. Mini-batch optimization enables 10- to 100-fold speed-up in large-scale ODE model parameter estimation. *Nature Communications*, 13:1–14, 2022.

[29] Leipzig University Database Group. Benchmark datasets for entity resolution. https://dbs.uni-leipzig.de/research/projects/benchmark-datasets-for-entity-resolution, accessed Nov. 2025.

[30] O. Binette et al. Entity-centric evaluation for record linkage and deduplication. *arXiv preprint* arXiv:2404.05622, 2024.

[31] S. Thirumuruganathan et al. A survey of entity resolution and record linkage. *Proc. VLDB Endowment*, 14(11), 2021.